IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Entrenar bien: sobreajuste y regularización

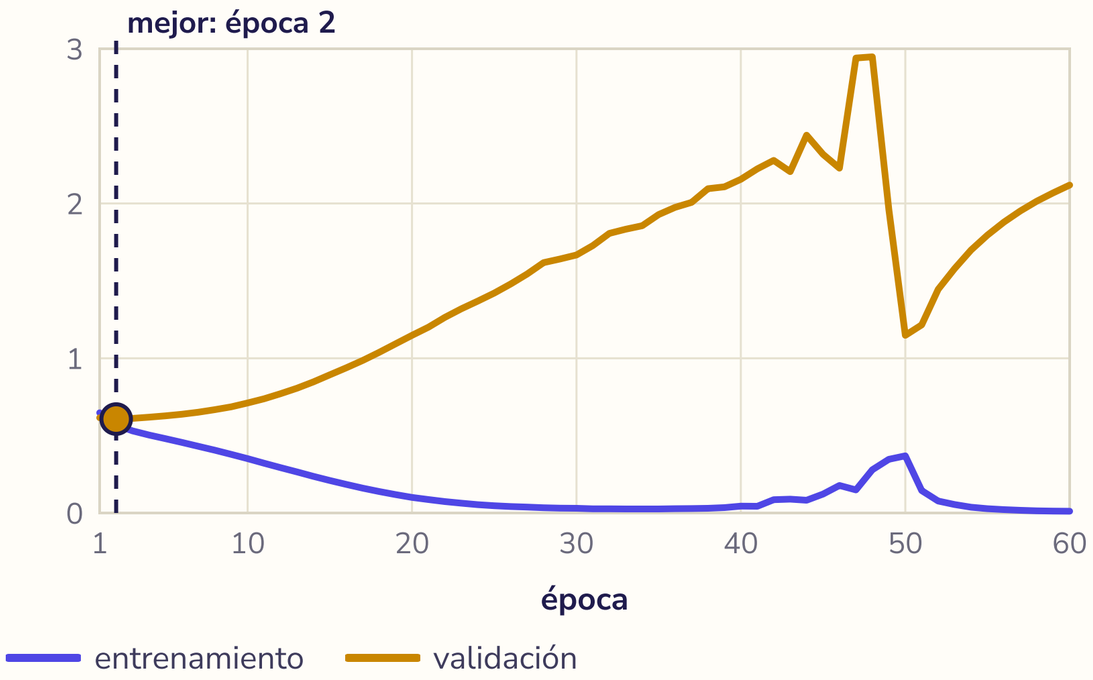
Curvas de aprendizaje, L2, dropout, parada temprana y callbacks

Sprint 10 · S10_03 · cuaderno: 55 min en clase (ejercicios 1 a 3)

LA IDEA · SECCIÓN 1

Una red que memoriza en lugar de aprender

Pérdida por época · red 256-128 con solo 800 reservas



42 241

parámetros de la red, para **800** reservas

0,011 vs 2,12

pérdida de entrenamiento al final frente a la de validación (mínimo: 0,607)

Brecha de generalización

La distancia entre las dos curvas.

LAS HERRAMIENTAS · SECCIÓN 2

Cuatro formas de frenar la memorización

Técnica	Idea en una frase	En Keras
L2 (weight decay)	Castiga los pesos grandes: la red prefiere soluciones «suaves»	`kernel_regularizer=keras.regularizers.l2(0.01)`
Dropout	En cada paso apaga al azar un % de neuronas: no puede depender de una sola	`keras.layers.Dropout(0.3)`
Parada temprana	Para cuando la validación deja de mejorar	callback `EarlyStopping`
Batch normalization	Reescala las activaciones en cada lote: entrena más estable y regulariza algo	`keras.layers.BatchNormalization()`

El remedio más barato suele ser más datos o un modelo más pequeño; estas herramientas reducen la brecha.

CÓMO ACTÚAN · L2 Y DROPOUT

L2 y dropout, con números pequeños

L2 suma una multa a la pérdida

pérdida total = error + λ · Σ w² con λ = 0,01

Dropout 0,4 apaga neuronas al azar

Las que siguen se multiplican por **1 / (1 − 0,4) = 1,67**.

Pesos de una neurona	Σ w²	Multa (λ = 0,01)
3 · −0,5 · 2	13,25	0,1325
1 · −0,5 · 2	5,25	0,0525

	n1	n2	n3	n4	n5
activación	0,8	1,5	0,2	1,0	0,6
¿se queda?	sí	**no**	sí	sí	**no**
al entrenar	1,33	0	0,33	1,67	0

El peso grande (3) es el que más multa pone, así que la red lo reduce. Dropout solo actúa al entrenar: al predecir se usan todas las neuronas.

CALLBACKS · SECCIÓN 3

Parar a tiempo y quedarse con lo mejor

Ejemplo con **patience=3**: la pérdida de validación por época.

época	1	2	3	4	5	6	7
pérdida val.	0,70	0,62	0,58	**0,57**	0,59	0,60	0,61
épocas sin mejorar	0	0	0	0	1	2	3

A la 7.ª para, y restore_best_weights=True devuelve los pesos de la época 4.

EarlyStopping

Para si la validación no mejora en **patience** épocas seguidas.

ReduceLROnPlateau

Si se estanca, baja la tasa de aprendizaje (**factor=0.5**: a la mitad).

ModelCheckpoint

Guarda en disco el mejor modelo (**save_best_only=True**).

Gradientes que desaparecen o explotan

Al retropropagar se **multiplica un factor por capa**: la sigmoide da **0,25 como máximo**; un peso de 1,5, ×1,5.

Capas	Factor 0,25 por capa: se apaga	Factor 1,5 por capa: se dispara
2	0,0625	2,25
5	0,00098	7,59
10	0,00000095	57,67
Síntoma	La pérdida apenas baja y las primeras capas casi no cambian	La pérdida salta de golpe o aparece **nan**

Remedio	En Keras
ReLU (derivada 1 si la entrada es positiva)	**activation="relu"**
Inicialización pensada para ReLU	**kernel_initializer="he_normal"**
Normalizar cada lote	**keras.layers.BatchNormalization()**
Recortar el gradiente	**keras.optimizers.Adam(clipnorm=1.0)**

EJERCICIO 1 · CUADERNO (CLASE)

¿En qué época empezó a memorizar?

```
perd_val = h_base.history["val_loss"]
perd_ent = h_base.history["loss"]
epoca_optima = int(np.argmin(perd_val)) + 1
brecha_final = perd_val[-1] - perd_ent[-1]
```

2

época de validación mínima, contando desde 1 (de 60)

2,11

brecha final = validación − entrenamiento

Época	1	2	5	10	20	40	60
Pérdida de entrenamiento	0,648	0,569	0,480	0,351	0,101	0,044	0,011
Pérdida de validación	0,616	**0,607**	0,628	0,711	1,148	2,156	2,119

Si la mejor época está muy por debajo del total, seguir entrenando solo empeoraba el modelo.

EJERCICIO 2 · CUADERNO (CLASE)

Cinco recetas contra el sobreajuste

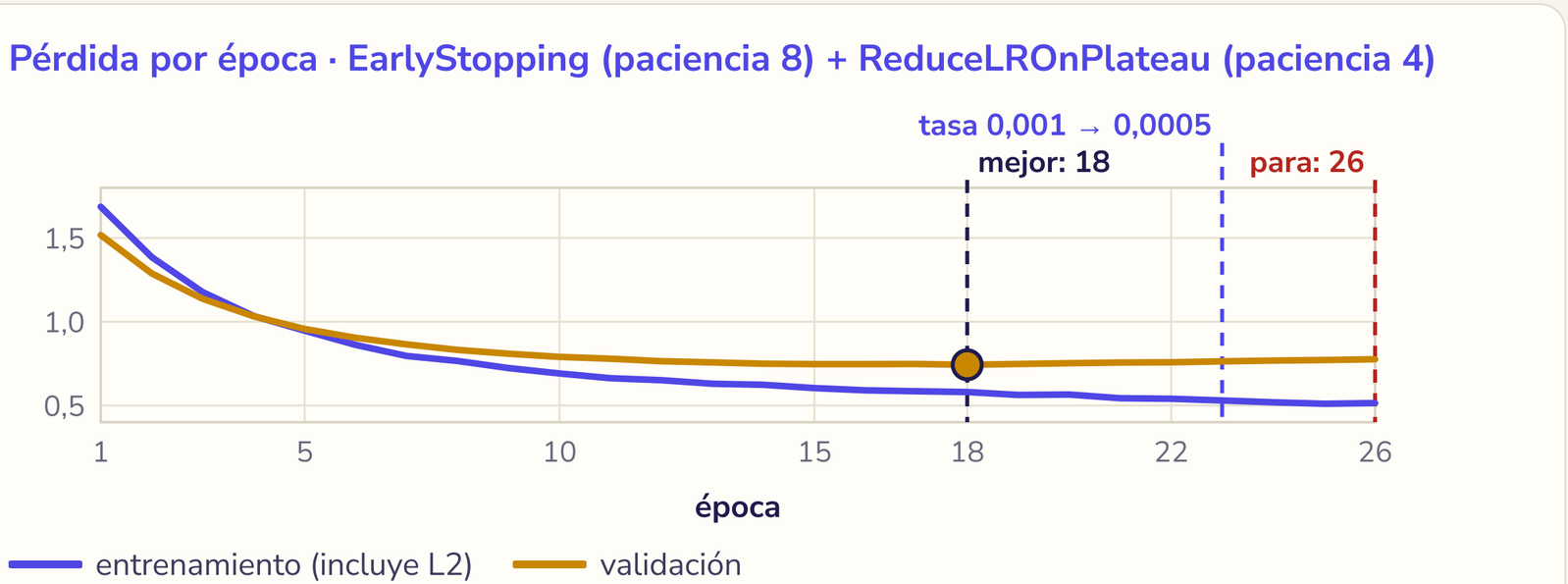
```
m_drop, h_drop = entrenar(A_peq, y_peq, epocas=60, dropout=0.4)
n_dropout = sum(isinstance(capa, keras.layers.Dropout) for capa in m_drop.layers)  # = 2
```

Receta (800 reservas, 60 épocas)	Mejor pérdida val.	Pérdida val. final	AUC val. final	AUC val. máximo
Sin regularizar	0,607	2,119	0,640	0,728
L2 = 0,01	0,696	1,008	0,667	0,731
Dropout 0,4	0,609	1,206	0,652	0,728
BatchNorm	0,645	1,509	0,672	0,701
L2 + dropout	0,743	0,936	0,683	0,731

Bajan la pérdida final (2,12 → 0,94–1,51), pero el AUC máximo apenas se mueve: no crean información.

EJERCICIO 3 · CUADERNO (CLASE)

Callbacks: la red para sola en la época 26



26 / 100

épocas entrenadas, de 100 posibles (18 + paciencia 8)

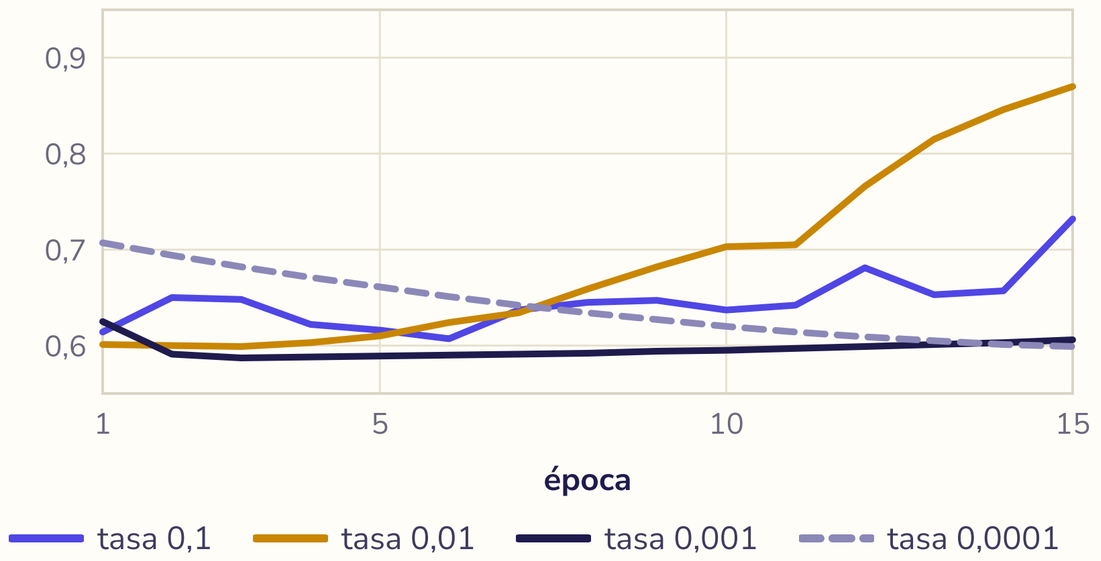
0,743

mejor pérdida de validación, en la época 18

```
mis_callbacks = [EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
                 ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4)]
```

SECCIÓN 4 · CASA

Tasa de aprendizaje y tamaño de lote



Prueba	AUC val. final	Segundos*
tasa 0,1	0,690	4,0
tasa 0,01	0,681	4,5
tasa 0,001	0,738	5,1
tasa 0,0001	0,737	4,0
lote 32	0,730	7,6
lote 128	0,742	5,1
lote 512	0,749	3,0

* Segundos en nuestra prueba, a título orientativo: en tu ordenador saldrán otros. 15 épocas por prueba.

EJERCICIO 4 Y MINI-RETO · CASA

Guardar el modelo y ganar a la red a ciegas

```
m_es.save(ruta_modelo)  # formato .keras
m_cargado = keras.models.load_model(ruta_modelo)
prob_cargado = m_cargado.predict(A_val, verbose=0)
```

Ojo con el preprocesado

El archivo **.keras** guarda arquitectura, pesos y optimizador, **no** el **ColumnTransformer**: guárdalo también (por ejemplo con **joblib**).

Red (3840 filas, 40 épocas, lote 64)	Pérdida val.
Grande 256-128 sin regularizar	1,334
64-32 + L2 0,003 + dropout 0,3 + parada (paciencia 6)	**0,615**

El archivo del modelo del ejemplo pesa unos **525 KB**.

Mini-reto: menos pérdida de validación que la red a ciegas. Se logra con una red más pequeña y las herramientas de hoy.

PUENTE AL RETO · S10_04 (DÍA 2)

De las curvas al veredicto del reto

1 · Misma partición

64 % entrenamiento, 16 % validación (decisiones y curvas) y 20 % prueba, **una sola vez**.

2 · Varias semillas

El MLP se entrena con **3 semillas**: media y variación, no un número suelto. Las curvas se leen con **diagnostico(...)**.

3 · Diferencia con incertidumbre

Si el intervalo de la diferencia de AUC incluye el 0, **no** se puede afirmar que uno sea mejor.

Ejemplo inventado	Diferencia de AUC (red – boosting)	Intervalo al 95 %	¿Incluye el 0?
A	+0,004	[–0,010 ; +0,018]	sí → no hay diferencia clara
B	+0,031	[+0,012 ; +0,050]	no → hay diferencia

Lo marcado «en casa» (tasa, lote, guardar, mini-reto) no es necesario para el reto.

HOY HAS APRENDIDO

Memorizar no es aprender

Diagnóstico

Sobreajuste = el entrenamiento mejora y la validación empeora. Se ve en las **curvas**, no en una cifra suelta

Remedios

L2, dropout y parada temprana reducen la brecha; más datos o una red más pequeña, aún mejor

Buenas prácticas

Callbacks para parar y guardar; tasa y lote se afinan mirando curvas; **.keras** más el preprocesado

Siguiente: S10_04 · el reto «¿Vale la pena la red?»